COMPUTER VISION

LAB SHEET :07

```
# Name: Deepti
# CUID:24250168
# Roll No.: 16   Sec:B

# Experiment No. 7
# Motion Estimation using Optical Flow Algorithms in Video Sequences

# Import required libraries
import cv2
import numpy as np

# Load the traffic video
video_path = "video.mp4"
cap = cv2.VideoCapture(video_path)

# Check whether the video was opened successfully
if not cap.isOpened():
    print("Error: Could not open video.mp4")
    print("Make sure video.mp4 is in the same folder as labsheet7.py")
    exit()

# Read the first frame
ret, old_frame = cap.read()

# Check whether the first frame was read
```

```
if not ret:
    print("Error: Could not read the first frame.")
    cap.release()
    exit()

# Convert the first frame to grayscale
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

# Shi-Tomasi Corner Detection parameters
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

# Detect feature points in the first frame
p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

# Check whether feature points were detected
if p0 is None:
    print("No feature points detected.")
    cap.release()
    exit()
```

```
# Lucas-Kanade Optical Flow parameters
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

# Create a mask for drawing trajectories
mask = np.zeros_like(old_frame)

# Process every frame of the video
while True:

    # Read the next frame
    ret, frame = cap.read()

    # Stop when the video ends
    if not ret:
        break

    # Convert current frame to grayscale
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
# Apply Lucas-Kanade Sparse Optical Flow
p1, st, err = cv2.calcOpticalFlowPyrLK(
    old_gray,
    frame_gray,
    p0,
    None,
    **lk_params
)

# Check whether optical flow was calculated
if p1 is not None:

    # Select successfully tracked points
    good_new = p1[st == 1]
    good_old = p0[st == 1]

    # Draw motion trajectories
    for new, old in zip(good_new, good_old):

        # Get coordinates
        a, b = new.ravel()
        c, d = old.ravel()

        # Convert coordinates to integers
        a, b = int(a), int(b)
        c, d = int(c), int(d)

        # Draw trajectory line
```

```
    mask = cv2.line(
        mask,
        (a, b),
        (c, d),
        (255, 255, 255),
        2
    )

    # Draw tracked feature point
    frame = cv2.circle(
        frame,
        (a, b),
        4,
        (0, 255, 0),
        -1
    )

    # Draw motion arrow
    frame = cv2.arrowedLine(
        frame,
        (c, d),
        (a, b),
        (0, 0, 255),
        2
    )

# Combine video frame and trajectories
output_lk = cv2.add(
```

```
    frame,
    mask
)

# Add title
cv2.putText(
    output_lk,
    "Lucas-Kanade Sparse Optical Flow",
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
    (0, 255, 0),
    2
)

# Display Lucas-Kanade result
cv2.imshow(
    "Lucas-Kanade Optical Flow",
    output_lk
)

# Update feature points
if len(good_new) > 0:
    p0 = good_new.reshape(-1, 1, 2)

# If too few points remain, detect new points
if len(good_new) < 10:
    p0 = cv2.goodFeaturesToTrack(
```

```
            frame_gray,
            mask=None,
            **feature_params
        )

# Apply Farneback Dense Optical Flow
flow = cv2.calcOpticalFlowFarneback(
    old_gray,
    frame_gray,
    None,
    0.5,
    3,
    15,
    3,
    5,
    1.2,
    0
)

# Calculate magnitude and angle of motion
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# Create HSV image for optical flow visualization
hsv = np.zeros_like(frame)
```

```
# Set saturation to maximum
hsv[..., 1] = 255

# Represent motion direction using hue
hsv[..., 0] = angle * 180 / np.pi / 2

# Represent motion magnitude using brightness
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert HSV optical flow to BGR
dense_flow = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)

# Add title
cv2.putText(
    dense_flow,
    "Farneback Dense Optical Flow",
    (10, 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.8,
```

```
    (255, 255, 255),
    2
)

# Display Farneback result
cv2.imshow(
    "Farneback Optical Flow",
    dense_flow
)

# Update previous grayscale frame
old_gray = frame_gray.copy()

# Wait for a short time
key = cv2.waitKey(30) & 0xFF

# Press ESC to exit
if key == 27:
    break

# Press R to reset feature points
if key == ord("r"):

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )
```

```
        mask = np.zeros_like(old_frame)

        print("Feature points reset.")

# Release the video
cap.release()

# Close all OpenCV windows
cv2.destroyAllWindows()

# Observation
# Lucas-Kanade Optical Flow tracks selected feature points between consecutive frames.
# It produces sparse motion information and is computationally efficient.
# Farneback Optical Flow estimates motion for almost every pixel in the frame.
# It provides dense motion information and visualizes both direction and magnitude.
# Traffic movement can be observed using both optical flow techniques.

# Question 1: What is Optical Flow? How is it used in computer vision?
# Optical Flow is the apparent motion of objects or pixels between consecutive video frames.
# It is used to estimate object movement, direction, speed, displacement, and motion patterns.

# Question 2: Explain the working principle of the Lucas-Kanade Optical Flow algorithm.
# Lucas-Kanade calculates the motion of selected feature points between consecutive frames.
# It uses image intensity changes and local image gradients to estimate the displacement
# of the feature points.
```

Question 3: What is Dense Optical Flow? How does it differ from Sparse Optical Flow?
Dense Optical Flow estimates motion for almost every pixel in an image.
Sparse Optical Flow estimates motion only for selected feature points.
Farneback is a Dense Optical Flow method, while Lucas-Kanade is commonly used
for Sparse Optical Flow.

Question 4: Compare the Lucas-Kanade and Farneback optical flow algorithms.
Lucas-Kanade tracks selected feature points and is generally faster.
Farneback calculates motion over a large number of pixels and provides more complete
motion information but requires more computation.

Question 5: What assumptions are made while computing optical flow?
Optical Flow assumes that brightness remains approximately constant between frames,
the motion between consecutive frames is relatively small, and nearby pixels have
similar motion.

Question 6: What factors can affect the accuracy of optical flow estimation?
Accuracy can be affected by object speed, illumination changes, camera movement,
motion blur, image noise, occlusion, large object displacement, and low-texture areas.

Question 7: Mention five real-world applications of optical flow in computer vision.
Five applications are autonomous driving, video surveillance, robotics,
gesture recognition, and sports analytics.

Question 8: Why are grayscale images generally used for optical flow computation?
Grayscale images provide intensity information needed for calculating image gradients.
They also reduce computation because only one channel needs to be processed.

Question 9: What are the limitations of optical flow algorithms in real-world environments?

Optical Flow can be affected by lighting changes, camera movement, occlusion,

motion blur, fast-moving objects, large displacement, and regions without strong features.

Question 10: How does optical flow contribute to applications such as autonomous driving,

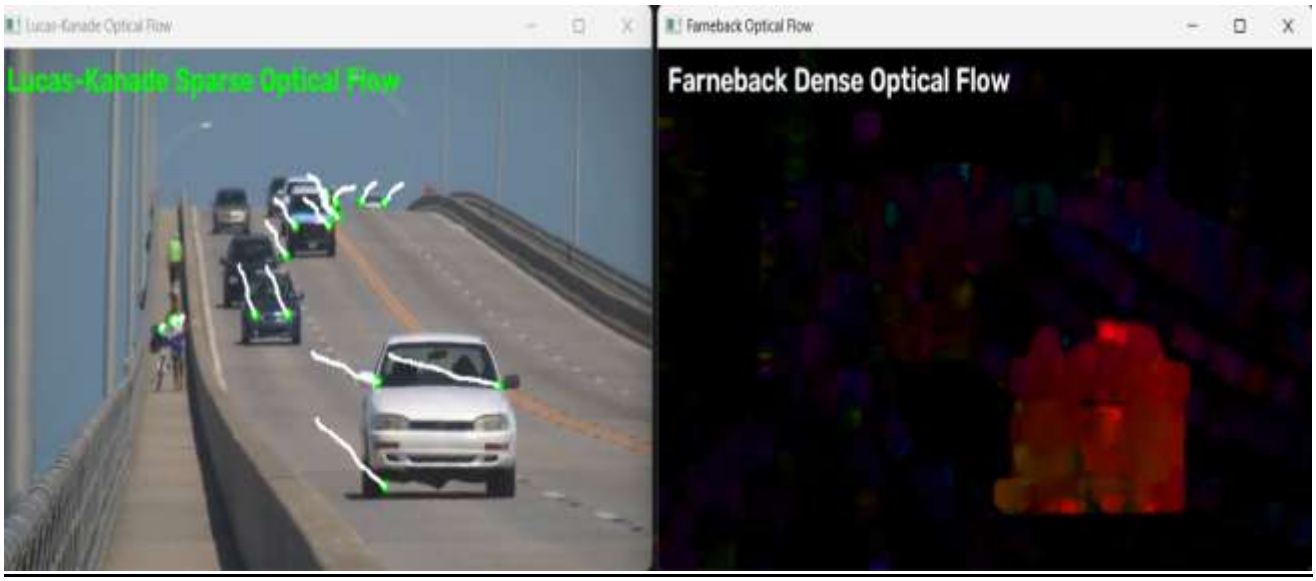
video surveillance, and action recognition?

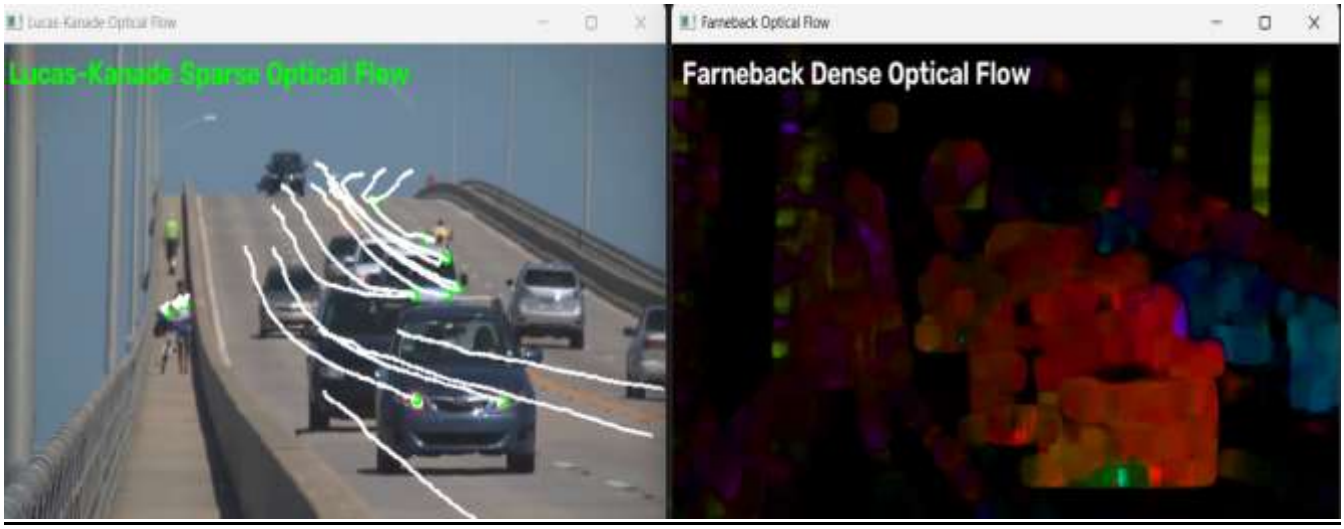
Optical Flow provides information about object movement and direction.

It helps autonomous vehicles understand moving objects, surveillance systems detect

movement, and action recognition systems analyze human motion.

OUTPUT


Lucas-Kanade Optical Flow
Lucas-Kanade Sparse Optical Flow
Farneback Optical Flow
Farneback Dense Optical Flow


Lucas-Kanade Optical Flow
Lucas-Kanade Sparse Optical Flow
Farneback Optical Flow
Farneback Dense Optical Flow